Quality, Architecture & Agile

IT Product Development

Innopolis University, 2026

Agenda

- Quality
- Architecture
- Agile

Product Quality

Quiz

What is a quality attribute?

Which quality attributes do you know?

Which quality attribute did the NavProject team skip?

Stakeholders and Quality

[Sbok5]:

Stakeholders are all individuals and groups affected by the Scrum project.

Your stakeholders are course instructors, your customer, peer teams, your team.

You should manage and meet their expectations about the quality of both your product and project (deliver on time, use appropriate processes, tools etc.)

Product Quality

[Iso25010]:

The **quality of a system** is the degree to which the system satisfies the stated and implied needs of its various **stakeholders**, and thus provides value.

Stakeholders' needs (functionality, performance, security, maintainability, etc.) are precisely what is represented in the quality model, which categorizes the **product quality** into characteristics and sub-characteristics.

> a.k.a "quality attributes" - properties of the system that stakeholders consider to measure quality

The ISO 25010 Quality Model

Quality → Characteristics → Sub-characteristics [Iso25010]:

SOFTWARE PRODUCT QUALITY								
FUNCTIONAL SUITABILITY	**PERFORMANCE EFFICIENCY**	**COMPATIBILITY**	**INTERACTION CAPABILITY**	**RELIABILITY**	**SECURITY**	**MAINTAINABILITY**	**FLEXIBILITY**	**SAFETY**
FUNCTIONAL COMPLETENESS FUNCTIONAL CORRECTNESS FUNCTIONAL APPROPRIATENESS iso25000.com	TIME BEHAVIOUR RESOURCE UTILIZATION CAPACITY	CO-EXISTENCE INTEROPERABILITY	APPROPRIATENESS RECOGNIZABILITY LEARNABILITY OPERABILITY USER ERROR PROTECTION USER ENGAGEMENT INCLUSIVITY USER ASSISTANCE SELF-DESCRIPTIVENESS	FAULTLESSNESS AVAILABILITY FAULT TOLERANCE RECOVERABILITY	CONFIDENTIALITY INTEGRITY NON-REPUDIATION ACCOUNTABILITY AUTHENTICITY RESISTANCE	MODULARITY REUSABILITY ANALYSABILITY MODIFIABILITY TESTABILITY	ADAPTABILITY SCALABILITY INSTALLABILITY REPLACEABILITY	OPERATIONAL CONSTRAINT RISK IDENTIFICATION FAIL SAFE HAZARD WARNING SAFE INTEGRATION

The Q42 Quality Model

[Q42]:

The arc42 quality model Q42 is a simple, pragmatic yet effective approach to **product and system quality**. It starts with **stakeholders' expectations** and requirements. From these, it derives 8 important system properties, which are sufficient to cover most, if not all, required, desirable or expected of the more than 100 *traditional* qualities.



How to Measure Your Product Quality? Idea 1

You ask stakeholders to assess the quality of a product version.

- **Pros:**
  - Stakeholders know their needs and thus are the perfect assessors.
- **Cons:**
  - It’s probably too late. You should’ve agreed on the quality in advance to have shared expectations and to build adequate quality into your product and processes [Pmbok7].

How to Measure Your Product Quality? Idea 2

You use quality attributes from an existing model.

- **Pros:**
  - Don’t need to design your own quality attributes.
- **Cons:**
  - Existing attributes are too abstract and can’t be directly measured. Example [Iso25010]:

    Capacity - Degree to which the maximum limits of a product or system parameter meet requirements.

How to Measure Your Product Quality? Idea 3

You agree with stakeholders on testable (pass/fail) quality requirements.

- **Pros:**
  - These requirements can be tested by all stakeholders (your team, your customer, etc.)
  - Their price can be estimated by tracing these requirements to related architectural decisions.
  - They can be put into a contract.
- **Cons:**
  - They may not cover the perceived quality [Garvin].

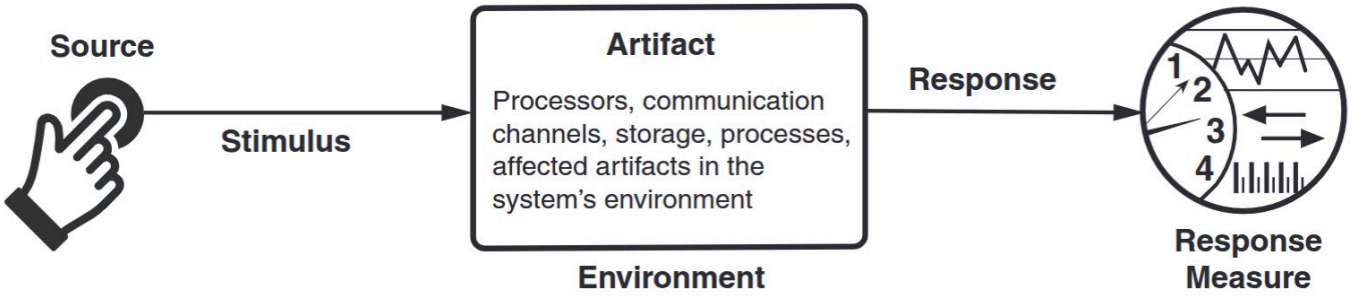
Quality Attribute Scenarios

QA scenarios are a way to write unambiguous testable quality requirements [Bck]. You should specify a realistic test to prove that a scenario is testable.

A scenario has 6 parts [QasArch]:

Part	Short description
Source of stimulus	An entity that generates the stimulus (human, external system, sensor, etc.)
Stimulus	A condition (event) to be considered when it arrives at a system
Environment	A system's condition when a stimulus occurs
Response	The activity undertaken at the arrival of the stimulus
Artifact	Some artifact that is stimulated; may be the whole system or part of it
Response measure	The response to the stimulus should be measurable someway so that the quality requirement can be tested

A General Quality Attribute Scenario for Availability



Source	Stimulus	Environment	Response	Response Measure
Internal/external: people, hardware, software, physical infrastructure, physical environment	Fault: omission, crash, incorrect timing, incorrect response	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation	Prevent the fault from becoming a failure Detect the fault Recover from the fault	Time or time interval when the system must be available Availability percentage Time to detect the fault

FIGURE 3.2 A general scenario for availability

Source: [Bck]

A QA Scenario for Moodle's Availability (AI-generated)

Source of Stimulus	External: A large number of students (e.g., 500+) simultaneously using the system.
Stimulus	Fault (Crash): An unexpected number of concurrent database write operations, triggered by all students submitting a high-stakes quiz in the final minute before the deadline, causes the primary database server to crash.
Artifact	The Moodle application, specifically the database server and the services that depend on it (e.g., quiz submission, gradebook, session management).
Environment	Normal (Peak) Operation: The system is operating under a heavy but expected load on a weekday evening, which is a known peak usage time for students.
Response	The system’s high-availability mechanism detects the primary database failure. It automatically initiates a failover to a redundant, standby database server. A brief maintenance page is displayed to users currently in the process of submitting, and the system then seamlessly restores their sessions once the failover is complete.
Response Measure	Full service is restored within 3 minutes. No more than 0.5% of student quiz submissions are lost and require resubmission. The system’s availability must not drop below 99.9% during the academic term.

A Test for the QA scenario (AI-Generated)

- **Step 1: Simulate the Stimulus**
  - In a staging environment, we use a load testing tool (JMeter) to simulate **500+ students** submitting a quiz at the exact same second.
- **Step 2: Inject the Fault**
  - At the precise moment of peak submission load, we intentionally **crash the primary database server** to trigger the automated failover system.
- **Step 3: Measure the Response**
  - The test is considered a **PASS** only if the system automatically recovers and meets these strict criteria:
    - **Recovery Time:** Full functionality is restored in **under 3 minutes**.
    - **Data Integrity:** Fewer than **0.5%** of student submissions are lost.
    - **Availability:** The overall uptime of **99.9%** is maintained.

How to get QA Scenarios?

- Quality Attribute Workshop (see Quality attribute workshop)
  - During QAW, facilitators identify and clarify architectural drivers with stakeholders and the architect and then help stakeholders generate, prioritize, and refine QA scenarios based on these drivers.
- Use an AI (similar to QAW; modify the steps below as necessary)
  - Load into the chat with the AI the existing product documentation, an existing quality model (ISO 25010, Q42, etc.) description, architectural drivers (see Architectural drivers), QAW procedure description.
  - Ask the AI to identify relevant quality attributes and come up with brief scenarios.
  - Ask stakeholders to validate and prioritize these scenarios.
  - Ask the AI to refine the most important scenarios.
  - Verify the requirements (consistency, feasibility, etc.)
  - Repeat steps as necessary.

Key Takeaways

- Elicit quality requirements at the project start to better manage customer expectations.
- Use QA scenarios to express testable quality requirements.
- Derive specific quality requirements from generic quality attributes in existing quality models.
- Use AI to produce requirements and specify corresponding tests.

More info: Quality

Architecture

Quiz

What is an architecture of a software system?

Which software systems have an architecture?

What is an Architecture of a Software System?

[Bck]:

The **software architecture** of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.

A **structure is architectural** if it supports reasoning about the system and the system's quality attributes. The reasoning should be about attributes of the system that are important to some stakeholder(s).

The attributes include **functionality** achieved by the system (see Functional Suitability in [Iso25010]).

Why is an Architecture Important?

[Bck]:

- The architecture connects (often abstract) business goals and requirements with the (concrete) tasks and implementation.
- Designing an architecture helps avoid costly changes later.
- An architecture will inhibit or enable a system’s driving quality attributes.
- A documented architecture enhances communication among stakeholders.
- The decisions made in an architecture allow you to reason about and manage change as the system evolves.
- An architecture defines a set of constraints on subsequent implementation.

What can Affect the Architecture?

[Bck]:

Architecturally Significant Requirements (ASRs) derived from:

- Functional requirements;
- Quality requirements;
- Business goals;
- Constraints (business, technical)

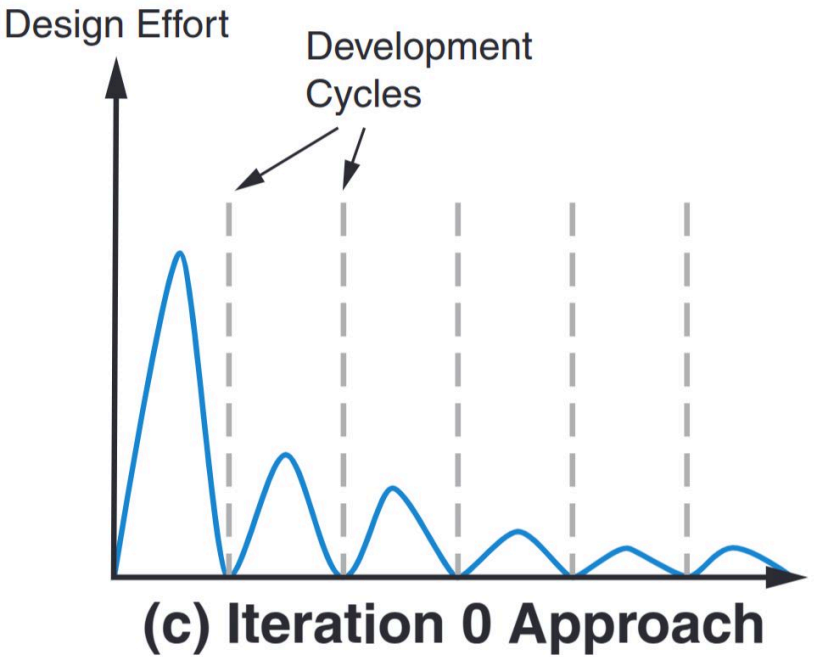
When to Design an Architecture?

[Bck]:

Design mostly at the very beginning of the project.

Continue designing and detailing the architecture throughout the project.

Change the architecture in response to changing requirements.



How to Design an Architecture?

See:

- Designing an Architecture [Bck, Ch. 20]
- An LLM-assisted approach to designing software architectures using ADD [LlmAdd]

How to Document the Architecture?

See [Bck, Chapter 22], Example 1, Example 2

In your assignments, you'll have to document at least these:

- Static views (context diagram, use case diagram, component diagram)
- Dynamic views (sequence diagram)
- Architectural decisions with justification

The diagrams must include a key (legend).

The documentation should be just barely good enough (of enough quality for the target audience at a given time).

Key Takeaways

- The architecture should mostly be developed early in the project.
- The architecture depends on architecturally significant requirements (ASRs).
- The ASRs should be gathered early in the project.
- The ASRs include quality requirements.
- Hence, quality requirements should be gathered early in the project.

More info: Architecture

Agile

Quiz

Which Agile approaches did you use?

Why do you think they were Agile approaches?

Agile Manifesto

[AgileManifesto]:

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Why use Agile for the ITPD Course?

ITPD is like a training before the Industrial Project (IP) course.

The IP lasts for two semesters. There will most probably be changes in requirements. You'll have to deliver value to real companies to succeed.

Agile works well for small self-organizing cross-functional teams. Agile is about managing changes and delivering value early. During ITPD, you'll practice both in a low-stakes project under our supervision.

We hope that the training during ITPD will help you succeed during the IP.

Agile Practices

During the course, you'll apply practices from Agile approaches to improve the quality of your system through processes and tools.

The focus will be on these approaches and associated practices/artifacts and deliverables (the list may be extended):

- Extreme Programming, or XP
  - Test-driven development: Tests
  - Continuous integration: CI/CD pipelines
  - Design improvement: Documented architecture
- Scrum [ScrumGuide]
  - Scrum Team, Scrum Events (Sprint Planning, Review, Retrospective): Sprint tracking sheet
  - Product Backlog, Sprint Backlog: GitHub Issues, GitHub Projects
  - Increments: Milestones and Releases on GitHub

You Won't use Scrum in ITPD

Please don’t tell during the Industrial Project that you were trained to use Scrum!

We teach you to apply Scrum practices, not the framework.

As explained in the [ScrumGuide] end note:

> “The Scrum framework, as outlined herein, is immutable. While implementing only parts of Scrum is possible, the result is not Scrum. Scrum exists only in its entirety and functions well as a container for other techniques, methodologies, and practices.”

Key Takeaways

- Scrum practices aren’t Scrum.
- We believe that Agile practices can improve the quality of your software.
- ITPD lets you train for the IP course.

More info: Agile, Scrum, Projects

References

1. [Sbok5]: SBOK® Guide, 5th ed., https://www.scrumstudy.com/sbokguide/download-free-buy-sbok#new-sbok-download
2. [Iso25010]: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010
3. [Pmbok6]: PMBOK® Guide, 6th ed., https://www.pmi.org/standards/pmbok
4. [Pmbok7]: PMBOK® Guide, 7th ed., https://www.pmi.org/standards/pmbok
5. [Q42]: https://quality.arc42.org/
6. [Garvin]: https://en.wikipedia.org/wiki/Eight_dimensions_of_quality
7. [QasArch]: Quality attributes of software architectures https://www.cs.unibo.it/cianca/wwwpages/as/5.pdf#page=26
8. [Bck]: L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed.
9. [LlmAdd]: An LLM-assisted approach to designing software architectures using ADD, https://arxiv.org/abs/2506.22688
10. [AgileManifesto]: Manifesto for Agile Software Development https://agilemanifesto.org/
11. [ScrumGuide]: Scrum Guide™ https://scrumguides.org/scrum-guide.html